VELLORE INSTITUTE OF TECHNOLOGY
(Deemed to be University under Section 3 of UGC Act, 1956)

REGEXLAB: AN INTERACTIVE REGULAR-EXPRESSION-TO-AUTOMATA COMPILER PIPELINE WITH VERIFIED NFA OPTIMIZATION

A Project Report submitted in partial fulfilment of the requirements for the course

BCSE307P – Compiler Design Project

by

Vignesh P C (24BCE0695)

Coppisetty Venkat Sai Ujwal (24BCE0667)

Under the guidance of

Dr. Islabudeen M
Associate Professor Grade 1, SCOPE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October 2026

DECLARATION

We hereby declare that the project report entitled **"REGEXLAB: AN INTERACTIVE REGULAR-EXPRESSION-TO-AUTOMATA COMPILER PIPELINE WITH VERIFIED NFA OPTIMIZATION"** submitted by us to Vellore Institute of Technology, Vellore in partial fulfilment of the requirement for the award of the degree of B.Tech. in Computer Science and Engineering, under the course BCSE307P, is a record of bona fide project work carried out by us under the guidance of Dr. Islabudeen M. We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: **<<TODO: submission date>>**

Vignesh P C (24BCE0695)

Coppisetty Venkat Sai Ujwal (24BCE0667)

ACKNOWLEDGEMENT

We owe a real debt of thanks to our project guide, **Dr. Islabudeen M**, Associate Professor Grade 1, School of Computer Science and Engineering (SCOPE), for his guidance, his patience through several design changes, and feedback that consistently pushed us to verify a claim rather than simply assert it.

We thank **Dr. Jaisankar N**, Dean, School of Computer Science and Engineering, for the infrastructure and the academic environment that made a project of this scope possible in the first place.

We are also grateful to the faculty of the Compiler Design course (BCSE307P) for a project format that let us learn lexical and syntax analysis by building something real with it instead of only reading about it, and to our families and friends for their patience and support while we were, more than once, up late arguing about whether a DFA state was really reachable.

TABLE OF CONTENTS

DECLARATION **2**
ACKNOWLEDGEMENT **3**
LIST OF FIGURES **5**
LIST OF TABLES **6**
LIST OF ABBREVIATIONS **7**
SYMBOLS AND NOTATIONS **8**
ABSTRACT **9**
1. INTRODUCTION **10**
1.1 Background 10
1.2 Motivations 10
1.3 Scope 10
2. PROJECT DESCRIPTION AND GOALS **11**
2.1 Literature Review 11
2.2 Research Gap 11
2.3 Objectives 11
2.4 Problem Statement 11
2.5 Project Plan 12
3. TECHNICAL SPECIFICATION **13**
3.1 Functional and Non-functional Requirements 13
3.2 Feasibility Study 13
3.3 Hardware and Software Specification 13
4. DESIGN APPROACH AND DETAILS **15**
4.1 System Architecture 15
4.2 Data Flow, Use Case, Class and Sequence Diagrams 15
5. METHODOLOGY AND TESTING **17**
5.1 Module Description, per Pass 17
5.2 Complexity and Resource Guards 17
5.3 Testing Strategy 18
6. PROJECT DEMONSTRATION **19**
7. RESULT AND DISCUSSION **20**
8. CONCLUSION AND FUTURE WORK **22**
9. REFERENCES **23**
APPENDIX A – SAMPLE CODE **24**

LIST OF FIGURES

Figure 4.1 – System architecture (layered) 8
Figure 4.2 – Data flow diagram (Level 1) 9
Figure 4.3 – Use case diagram 10
Figure 4.4 – Class diagram (core data contracts) 10
Figure 4.5 – Sequence diagram for one compile-and-match request 11
Figure 5.1 – Pass pipeline for pattern (a|b)*abb 13

Note: the figures in this report are schematic diagrams, drawn directly from the project's own source identifiers (file, type, and function names checked against the code), not graphviz or matplotlib output. We have **not yet captured** live browser screenshots of the running application (the AST/NFA/DFA/min-DFA views, step playback, the Growth Panel, the optimizer's safe and unsafe witness view). That needs a running dev server and a real browser session, which we plan to do before this report is finalised, and we would rather note the gap here than fake it.

LIST OF TABLES

Table 3.1 – Functional requirements 6
Table 3.2 – Non-functional requirements 7
Table 3.3 – Hardware and software specification 7
Table 5.1 – Module description per compiler pass 12
Table 5.2 – Complexity of each pass 14
Table 5.3 – Input guards 14
Table 5.4 – Automated test results 15
Table 5.4b – Compiled-WebAssembly limit-probe suite 15
Table 7.1 – Pipeline state counts for representative patterns 17
Table 7.2 – Minimization stress timings, native build 18
Table 7.2b – Same three patterns, rebuilt WASM module 18
Table 7.3 – Bounded-optimizer safe vs. unsafe deletion examples 18
Table 7.4 – Cost analysis (tooling) 19

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ASCII	American Standard Code for Information Interchange
AST	Abstract Syntax Tree
CI	Continuous Integration
CMake	Cross-platform Make (build-configuration tool)
CSS	Cascading Style Sheets
DFA	Deterministic Finite Automaton
DFD	Data Flow Diagram
HTML	Hypertext Markup Language
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LALR	Look-Ahead Left-to-right Rightmost-derivation (parser class)
LL(1)	Left-to-right, Leftmost-derivation, 1 symbol of lookahead
NFA	Nondeterministic Finite Automaton
PR	Pull Request
SCOPE	School of Computer Science and Engineering
SVG	Scalable Vector Graphics
TOC	Table of Contents
TS	TypeScript
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
WASM	WebAssembly

SYMBOLS AND NOTATIONS

Symbol	Meaning
Σ	Input alphabet (the set of symbols the automaton reads; ASCII bytes 0x00–0x7F in this project)
Q	Finite set of states of an automaton
q_0	Start (initial) state, $q_0 \in Q$
F	Set of accepting (final) states, $F \subseteq Q$
δ	Transition function, $\delta: Q \times \Sigma \to Q$ (DFA) or $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ (NFA)
ε	The empty string; also labels a "no input consumed" transition in an NFA
$L(M)$	The language (set of accepted strings) recognized by automaton M
$\|Q\|$	Number of states (state count)
$2^{\|Q\|}$	Powerset ceiling – the theoretical maximum number of DFA states subset construction could produce from an NFA of $\|Q\|$ states
$O(\cdot)$	Big-O asymptotic time/space complexity notation
$\perp$	The rejecting sink (dead) state shown by the visualizer when a DFA/min-DFA transition is missing

ABSTRACT

Most programmers use a regular expression every week and never see what actually happens underneath it. That gap is exactly what RegexLab sets out to close. It is a hand-built, dependency-free regular expression engine that compiles a pattern through a visible, step-by-step pipeline rather than hiding everything behind a single match or no-match answer. A pattern is parsed into an abstract syntax tree, turned into a nondeterministic finite automaton by Thompson's construction, determinized into a DFA by subset construction, and reduced to its unique minimal form by Moore's algorithm, and every one of those intermediate structures is kept around and shown to the learner, not discarded once the final answer is ready.

We built the engine twice on purpose. A TypeScript version came first and still lives in the repository as the design-validation prototype, while the production engine is written in C++20 and compiled to WebAssembly so it runs directly in the browser. Both versions are checked against two independent, well-trusted reference engines, native C++'s `std::regex` and JavaScript's built-in `RegExp`, across thousands of randomly generated patterns and inputs. A real bug would have to slip past both languages and both test frameworks at once to go unnoticed, which is a much stronger bar than trusting either engine alone. On top of that core pipeline we added two features that go beyond the standard construction: a Subset Growth / NFA Structure Lab that reports raw, reachable, and minimized state counts alongside the theoretical powerset ceiling, and a Bounded Optimization Explorer that searches a strictly bounded space of epsilon-edge deletions on the Thompson NFA and proves, by product-construction equivalence checking, which ones are safe. Every candidate it cannot prove safe comes with a concrete witness string showing exactly where the language changed, and when a configured resource limit is hit before a proof completes, it says "inconclusive" instead of guessing.

This report walks through the system's architecture, maps it onto the compiler-design syllabus (lexical analysis through the indirect regex-to-DFA route, syntax analysis through a hand-written recursive-descent parser, code generation through the matcher and the WebAssembly build, DFA minimization standing in for the optimization phase), and lays out its resource-safety guards on pattern length, nesting depth, NFA state count, and minimization work. It documents the automated test results, 209 Vitest cases across 20 files and 81 native C++ Catch2 cases totalling 4,923 assertions, all passing, and a real regression we found and fixed while reviewing the project's own pull request history: a minimization-pass bug that had quietly dropped an alphabet-compression optimization and wrongly rejected a perfectly legal pattern as too complex, along with the timing evidence confirming the fix. We are just as upfront about what the project does not do. Only the indirect route to a DFA is implemented, not the direct method. Only Moore's minimization is implemented, not Hopcroft's. Matching is full-string only, the alphabet is ASCII only, and the bounded optimizer gives a verified-safe reduction within its search bound, not a provably minimum-size NFA.

(Word count: 495 words)

CHAPTER 1
INTRODUCTION

1.1 Background

A regular expression engine is one of the most heavily used pieces of compiler technology that a working programmer never actually sees the inside of. It is also, conveniently, the worked example that the course's prescribed textbook uses to introduce lexical analysis: Aho, Lam, Sethi and Ullman's **Compilers: Principles, Techniques, and Tools** (2nd ed.) walks through turning a regular expression into a syntax tree, the syntax tree into a nondeterministic finite automaton (NFA) by Thompson's construction, the NFA into a deterministic finite automaton (DFA) by subset construction (independently described by Rabin and Scott), and finally reducing that DFA to its unique minimal form by a state-equivalence algorithm such as Moore's or Hopcroft's. RegexLab builds exactly this pipeline by hand, with no regular-expression library and no automata-theory package anywhere in the engine, and keeps every intermediate structure visible to whoever is using it.

1.2 Motivations

Three things pulled us toward this project. The first was simply wanting to see automata theory move: it is normally taught on paper, with small NFAs and DFAs drawn by hand, and we wanted a tool that performs the same constructions on real input and lets a learner step through them the way a debugger steps through code. The second was correctness as a discipline in its own right. Building the same algorithms twice, once in TypeScript and once in C++, and checking both against independent reference engines turned the project into a genuine exercise in differential testing, not just parsing. The third, said plainly, was that this project gave us something concrete to point to. "Implemented Thompson's construction, subset construction, and Moore's minimization from scratch" is a claim we can actually back up line by line in the code, and the WebAssembly compilation step is a second, literal code-generation stage on top of the first.

1.3 Scope

In scope: a hand-written recursive-descent parser for an extended regular-expression grammar, covering literal characters with backslash escaping, concatenation, alternation, the Kleene star, one-or-more and zero-or-one quantifiers, parenthesised grouping, character classes, the wildcard `.`, and the shorthand classes `\d`, `\w`, `\s` and their negations. Also in scope: Thompson's construction, subset construction, Moore's DFA minimization, a DFA-driven matcher, a hand-rolled SVG automaton visualizer with step-by-step playback, a Subset Growth / NFA Structure Lab that exposes structural automaton measurements, and a Bounded Optimization Explorer that searches a strictly bounded space of epsilon-edge deletions and verifies every candidate's language equivalence.

Out of scope, and left out on purpose rather than forgotten: the direct DFA-construction method (nullable, firstpos, lastpos, followpos), Hopcroft's faster minimization algorithm, substring or partial matching, and backreferences or lookahead/lookbehind, since both of the last two break the regular-language guarantee the whole project rests on. We have also been careful never to describe the bounded optimizer as finding a provably minimum-size NFA. What it finds is a verified-safe reduction within its configured search bound, and that distinction matters.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

This project rests on a handful of classical results. Kleene's characterization of regular languages is made constructive by **Thompson's construction** [1], which builds an NFA from a regular expression in time linear in the expression's size using a small set of fixed per-operator wiring rules. **Rabin and Scott** [2] gave the subset-construction algorithm that determinizes any NFA into an equivalent DFA, at the cost of a worst-case exponential blow-up in state count. We say that plainly in this report rather than quietly stepping around it. **Moore** [3] gave an iterative partition-refinement algorithm for collapsing a DFA to its unique minimal form, and **Hopcroft** [4] later improved on it asymptotically, $O(n \cdot \log n \cdot k)$ instead of Moore's $O(n^2 \cdot k)$, by always refining from the smaller half of a split. That last sentence is a fact about the literature, not a claim about this project: **RegexLab implements Moore's algorithm only** (see §5.2 and §8), and Hopcroft's was deliberately left out. **Aho, Lam, Sethi and Ullman** [5] is the textbook that frames all of this as the worked example for a course's lexical-analysis module, and it is the explicit syllabus anchor behind this project's design.

Outside the purely theoretical line, **Cox** [6] explains from an engineering angle why real-world regex libraries built on backtracking search can blow up to exponential time on adversarial input, and makes the case for automaton-based matching, the approach this project takes, as the principled way around that problem. More recent work keeps pushing on how hard these constructions really are. **Baburin and Cotterell** [7] show that estimating how much subset construction's exponential blow-up will actually cost is itself PSPACE-hard, which is a sharper and more recent version of exactly the honesty this report tries to practise around §2.1's own exponential-blow-up admission. **Jiang and Ravikumar** [8] prove that finding a minimum-size NFA for a given regular language is PSPACE-complete, and **Clemente and Mayr** [9] give practical algorithms for reducing an NFA's size while provably preserving its language, which is the same safety guarantee the Bounded Optimization Explorer (§2.3, §7.3) is built around, just at a scale this project's bounded search does not attempt. These sources place the project's algorithms in context; we are not claiming RegexLab contributes a new result to that literature.

2.2 Research Gap

To our knowledge, and we have looked, we have not found an openly built, dependency-free, two-language regular-expression compiler pipeline that is built specifically for teaching in the way RegexLab is: one that exposes every pass's intermediate output to an interactive visualizer and also includes a bounded, verified NFA-structure optimizer that hands back a concrete counterexample whenever it proves a simplification unsafe. Most teaching implementations of these algorithms that we came across are standalone scripts or notebook exercises that produce one final diagram and stop there, not a tested, dual-implementation, browser-deployed pipeline with an automated and safety-verified optimization step built in.

2.3 Objectives

We set out with six concrete goals. Implement Thompson's construction, subset construction, and Moore's minimization from scratch, with no automata or regex library, in two independent language implementations. Validate both implementations by differential testing against independent reference engines. Expose every pass's output through an interactive, hand-rolled SVG visualizer. Add a bounded, formally-verified NFA-structure exploration feature, the Bounded Optimization Explorer, that never reports a false "safe" simplification. Compile the production engine to WebAssembly and verify its resource guards directly in a real browser build. And document the project's syllabus mapping, complexity analysis, and limitations honestly, including the parts where the mapping is thin.

2.4 Problem Statement

Given a regular expression from the supported extended grammar and a candidate input string, the engine has two jobs. First, decide whether the input belongs to the language the expression describes, using only hand-built automata-theoretic machinery and within explicit, enforced bounds on pattern length, nesting depth, NFA state count, and minimization work. Second, given a bounded set of structural simplifications to the expression's Thompson NFA (specifically, epsilon-edge deletions), decide whether each one preserves the recognized language exactly. That second answer has to come from product-construction equivalence

checking, not from a heuristic or from sampling, and whenever the answer is no, the engine has to produce a concrete, shortest distinguishing input.

2.5 Project Plan

We planned the project in phases and tracked progress against a living checklist in our own internal working-agreement document. The Core phase covered literal characters, concatenation, alternation, the Kleene star, grouping, the full five-pass pipeline in both languages, golden-value and differential tests, the WebAssembly binding, the visualizer, and Moore's minimization. Stretch A added the `+` and `?` quantifiers, and Stretch B added `[a-z]`-style character classes. A Hard Stretch tier, Hopcroft's minimization and isomorphism-based equivalence testing, was explicitly locked and never started, since we had agreed it would only be worth attempting once both of us could explain the simpler Moore's-algorithm version from memory, unaided. The Subset Growth / NFA Structure Lab and the Bounded Optimization Explorer came later, as a deliberately scoped addition once the Core was already working end to end.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Functional and Non-functional Requirements

Table 3.1 – Functional requirements

ID	Requirement
FR1	Parse a supported extended regular-expression grammar into an abstract syntax tree, reporting a structured, positioned error for any invalid pattern.
FR2	Construct an NFA from the AST using Thompson's construction.
FR3	Construct a DFA from the NFA using subset construction.
FR4	Minimize the DFA using Moore's partition-refinement algorithm.
FR5	Match a given input string against the minimized DFA and produce a step-by-step trace.
FR6	Render the AST, NFA, DFA, and minimized DFA as an SVG graph with playback controls.
FR7	Report structural automaton measurements (raw/reachable/minimized state counts, powerset ceiling).
FR8	Search a bounded space of epsilon-edge-deletion candidates on the NFA and verify each candidate's language equivalence against the baseline, reporting a distinguishing witness for every unsafe candidate.

Table 3.2 – Non-functional requirements

ID	Requirement
NFR1	Zero regular-expression or automata-theory runtime dependencies in the engine itself.
NFR2	Every resource-intensive pass (Thompson's construction, subset construction, minimization) must fail fast with a typed error rather than hang, once an explicit, documented budget is exceeded.
NFR3	The production engine must run client-side in a browser via WebAssembly, with no server-side regex evaluation.
NFR4	Both language implementations (TypeScript, C++) must be validated against independent reference engines by differential testing.

3.2 Feasibility Study

Technical feasibility: every algorithm we used (Thompson's construction, subset construction, Moore's minimization, product-construction equivalence checking) is well-documented and textbook-level, so there was no open research risk. The one real technical risk we ran into was unbounded recursion and state growth on adversarial input, and we closed it with explicit guards on pattern length, nesting depth, NFA state count, and minimization work.

Economic feasibility: the whole toolchain is free and open source: GCC/Clang, CMake, Node.js, TypeScript, Vitest, Catch2, Emscripten, React, and Vite. The project carries no licensing cost at all (see Table 7.4).

Social feasibility: this is a self-contained teaching tool. It collects no personal data, tracks nothing, and does not depend on any paid external service for its core engine. The only optional piece that would reach outside the browser is an AI-assisted suggestion layer, which is still only planned and not required for the core pipeline to work, and even that is designed so any suggestion it makes has to pass back through the same hand-built parser before a user ever sees it.

3.3 Hardware and Software Specification

Table 3.3 – Hardware and software specification

Category	Specification
Development hardware	Any x86-64 machine capable of running Node.js and a C++20 compiler; no GPU or specialised hardware required.

Operating system (development)	Windows 11 with a WSL2 Ubuntu environment (as used for this report's own verification run); Linux and macOS are equally supported.
Core engine language	C++20 (production, compiled to WebAssembly via Emscripten) and TypeScript (strict mode, prototype/design-validation implementation).
Build tooling	CMake ≥ 3.16, a C++20-capable compiler (GCC 15.2 used for this report's verification build), Node.js 22, npm.
Test tooling	Vitest (TypeScript), Catch2 v3 (C++, fetched via CMake FetchContent).
Frontend	React, Vite, TypeScript, hand-rolled SVG (no charting/graph library).
Deployment target	Any modern browser capable of running a WebAssembly module (Emscripten `MODULARIZE`/`EXPORT_ES6` build).

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The system is organised into three strictly decoupled layers (Figure 4.1). The **Core Engine** is a pure pass pipeline: a pattern and an input string go in, structured JSON comes out, and it has no idea a UI or an AI layer even exists. The **Visualization Layer** is a pure function of the engine's own output, with no logic of its own. The **Suggestion Layer** is optional and separately gated: it only switches on when parsing fails, and any candidate fix it proposes has to pass back through the same hand-built parser before it is ever shown to a user.

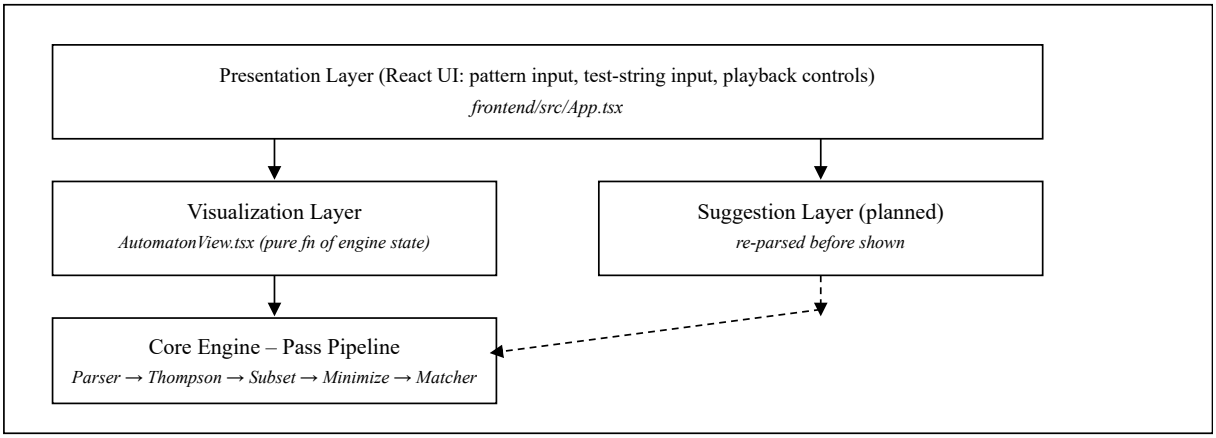


Figure 4.1 – System architecture (layered)

4.2 Data Flow, Use Case, Class and Sequence Diagrams

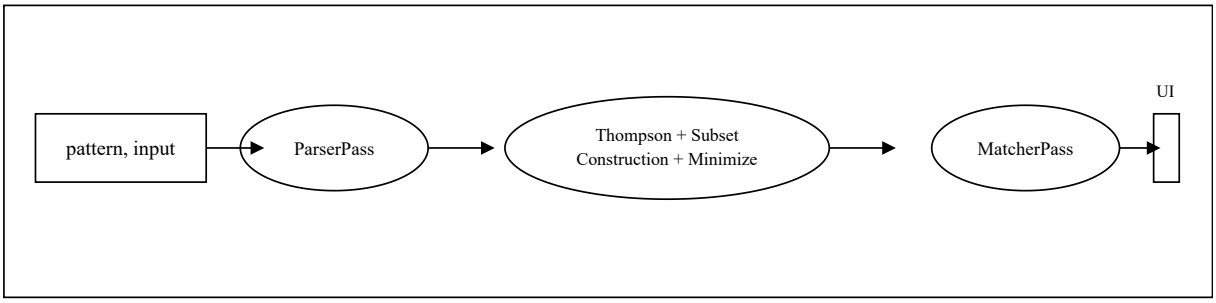


Figure 4.2 – Data flow diagram (Level 1)

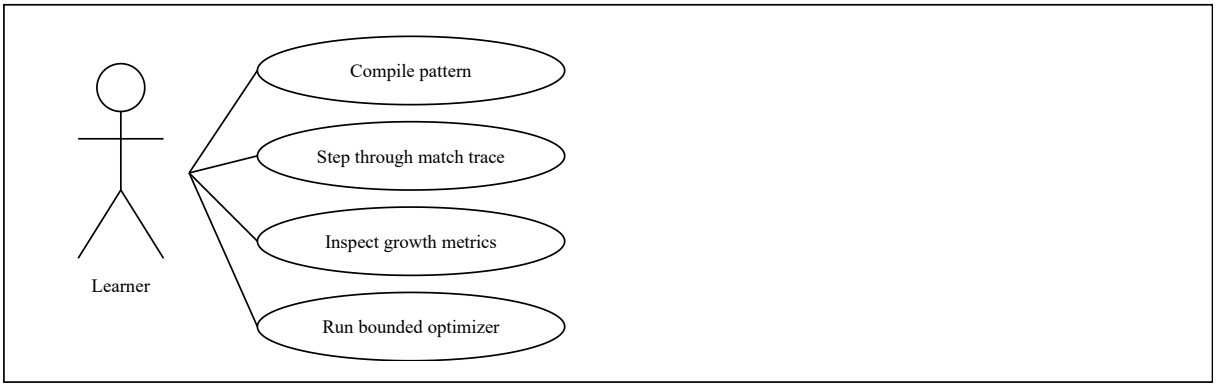


Figure 4.3 – Use case diagram

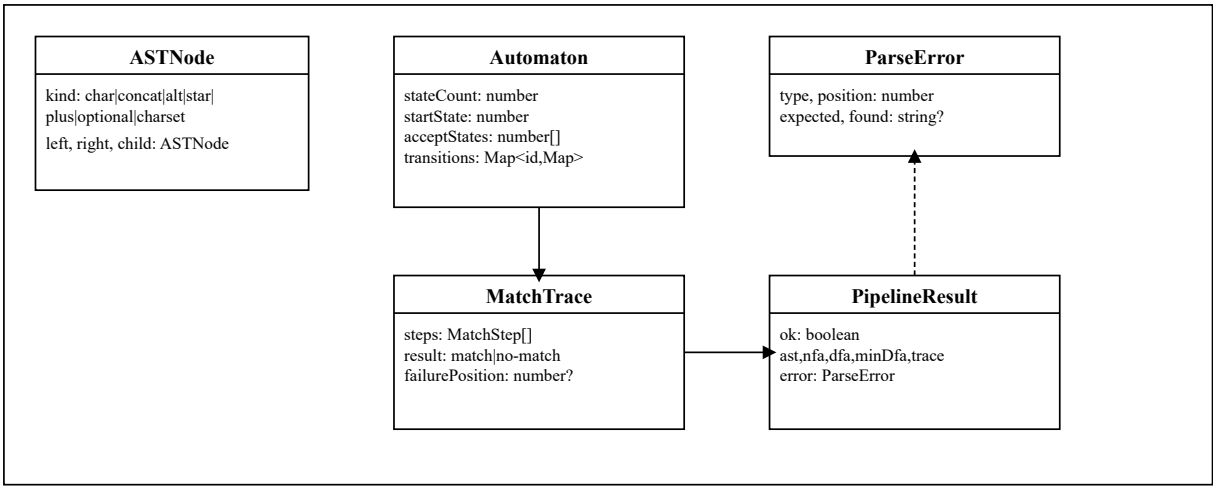


Figure 4.4 – Class diagram of the core data contracts (field names verified against `engine/include/regexlab/types.hpp` and `frontend/src/types/engine.ts`)

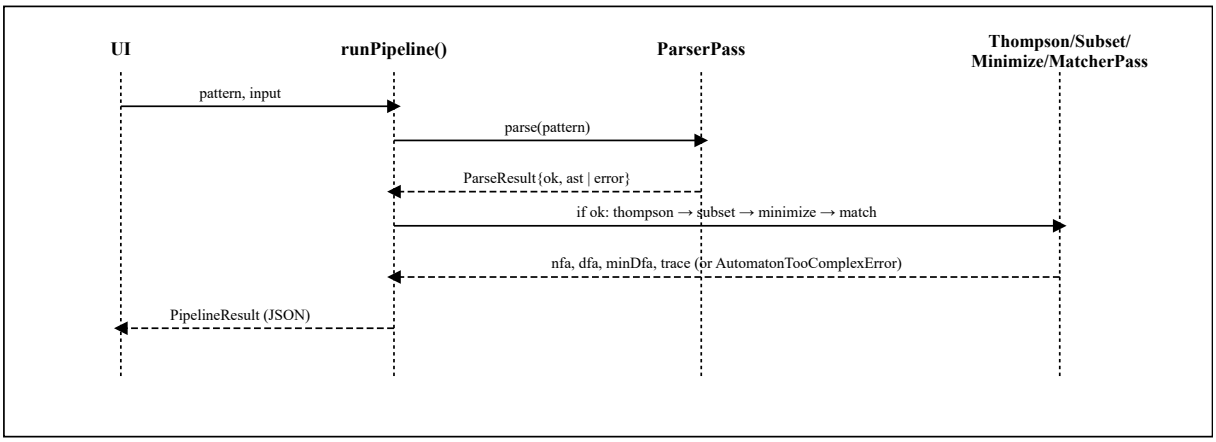


Figure 4.5 – Sequence diagram for one compile-and-match request (`engine/pipeline.cpp:runPipeline`)

CHAPTER 5
METHODOLOGY AND TESTING

5.1 Module description, per pass

Table 5.1 – Module description per compiler pass

Pass	Algorithm	Input → Output	Source files
ParserPass	Hand-written recursive descent (precedence by call-chain: regex → term → factor → atom)	pattern string → AST or ParseError	`engine/passes/parser-pass.ts`, `.cpp`
ThompsonPass	Thompson's construction [1]	AST → NFA	`engine/passes/thompson-pass.ts`, `.cpp`
SubsetConstructionPass	Subset/powerset construction [2], worklist-framed	NFA → DFA	`engine/passes/subset-construction-pass.ts`, `.cpp`
MinimizationPass	Moore's algorithm [3] (partition refinement), with alphabet equivalence-class compression	DFA → minimal DFA	`engine/passes/minimization-pass.ts`, `.cpp`
MatcherPass	Direct DFA simulation	minimal DFA, input string → MatchTrace	`engine/passes/matcher-pass.ts`, `.cpp`

Worth stating plainly: the five passes are plain functions chained together by `pipeline.cpp`, not instances of some shared `Pass<In,Out>` interface. An early design sketch floated that kind of interface, but the shipped code never ended up using it, and we would rather say so than describe the code as more abstract than it actually is.

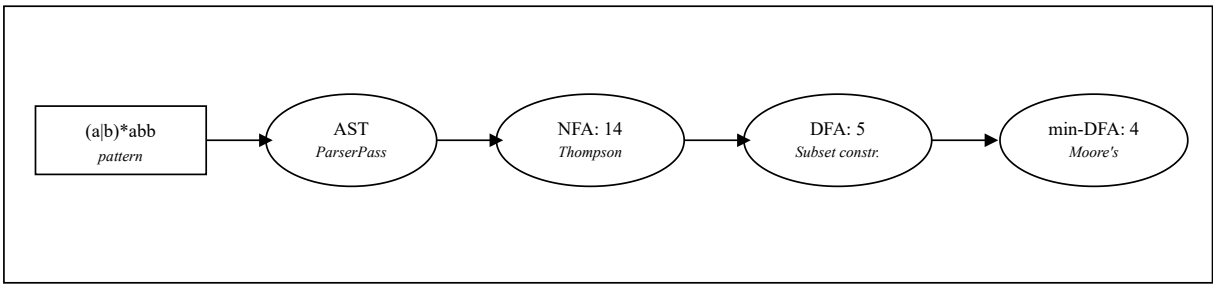


Figure 5.1 – Pass pipeline for pattern `(a|b)*abb`, with the actual state counts from Table 7.1

5.2 Complexity and resource guards

Table 5.2 – Complexity of each pass (n = DFA/NFA state count as noted, k = alphabet size, m = NFA state count)

Stage	Time	Space	Note
Parsing	O(n)	O(n)	n = pattern length
Thompson's construction	O(n)	O(n)	linear in AST size; a saturating state-count estimate is checked before allocation
Subset construction	$O(2^m)$ worst case	$O(2^m)$ worst case	the exponential blow-up is named, not hidden
Minimization (Moore's)	$O(n^2 \cdot k)$	$O(n \cdot k)$	k measured after alphabet equivalence-class compression; see §7.2 for a regression found and fixed in this project's own review process
Matching	O(n) per string	O(1) active state	why DFA matching avoids the exponential worst case of naive backtracking engines [6]

Table 5.3 – Input guards (verified directly against the source)

Guard	Value	Behaviour
Pattern length	500 characters	`PatternTooComplex` at position 500, over an all-ASCII prefix
Nesting depth	1000 levels of `(`	`PatternTooComplex` at position 1000; mathematically unreachable under the 500-char cap, kept as defense in depth
Thompson NFA size	2,048 states	`AutomatonTooComplexError` before allocation, via a saturating state-count estimate
Minimization input size	20,000 DFA states (default)	a cheap up-front size check, before any alphabet work begins
Minimization work	5,000,000 work units (default)	computed as state-count² × alphabet size, measured **after** alphabet equivalence-class compression
Alphabet	ASCII only	a non-ASCII byte in the pattern is an `UnexpectedToken` at its own position

5.3 Testing strategy

Testing happens in layers. Golden-value unit tests cover each pass individually, ported one-to-one between the two languages. A pipeline-equivalence test confirms that calling the chained pipeline gives the same result as manually chaining the five passes yourself. Differential testing runs the C++ engine against `std::regex` and the TypeScript engine against native, anchored `RegExp` over thousands of randomly generated pattern-and-input pairs. The C++ build compiles with strict warnings (`-Wall -Wextra`) enabled. We also practise mutation discipline: deliberately break something, confirm the relevant test actually goes red, then revert it, so we know the tests are testing something real. And every resource guard has its own test that configures a deliberately tiny budget to confirm the guard actually fires rather than silently doing nothing.

Table 5.4 – Automated test results (this report's own verification run, PR #72 head plus a local minimization-pass fix – see §7.2)

Suite	Result
TypeScript strict typecheck (`tsc --noEmit`)	Passed, zero errors
Vitest	209 tests passed across 20 test files, 0 failed
Native C++ Catch2 suite (Release build, `engine_tests`)	81 test cases passed, 4,923 assertions passed, 0 failed
Frontend production build (`npm --prefix frontend run build`)	Passed – 263.84 kB JS (82.01 kB gzip), 18.46 kB CSS (4.23 kB gzip)

Table 5.4b – Compiled-WebAssembly limit-probe suite (`scripts/check-wasm-limits.mjs`), rebuilt WASM module (Emscripten 6.0.11, Release)

Result	Value
Probes passed	31 / 31
Acceptance / pattern-length-cap / nested-plus / nesting-depth / non-ASCII / length-parity probes	27 (unchanged from the pre-fix tree)
Restored timed stress probes (§7.2 fix)	4, all passing well under their time budgets

The WASM target was rebuilt from source (Emscripten 6.0.11, Release) and `scripts/check-wasm-limits.mjs` was run against the fresh module for this report: **31/31 probes passed**, including the three previously-regressed stress probes and this PR's own nested-plus guard probes. See Table 7.2 for the individual stress-probe timings.

CHAPTER 6
PROJECT DEMONSTRATION

A typical walkthrough starts with a learner typing in a pattern, say `(a|b)*abb`, and a test string, say `ababb`. The engine parses the pattern into an AST, builds the 14-state Thompson NFA, determinizes it into a 5-state DFA, minimizes that down to a 4-state DFA by Moore's algorithm, and runs the test string through it, producing a step-by-step trace that ends in acceptance (these exact numbers are verified in Table 7.1, not approximated). The visualizer draws the AST, NFA, DFA, and minimized DFA as SVG graphs with playback controls that light up the active state or states as each character is consumed. If a character has nowhere to go in the DFA or minimized DFA view, the visualizer shows an explicit rejecting sink state and highlights the character that failed. The NFA view is left without a fabricated sink on purpose, since a real NFA has partial transitions by nature and inventing a sink there would misrepresent it.

A second walkthrough shows the Subset Growth / NFA Structure Lab. For a given pattern, the panel reports the raw NFA state count, how many states subset construction actually reaches, and the minimized DFA's state count, side by side with the theoretical powerset ceiling of $2^{|Q|}$. Seeing the gap between that ceiling and what actually gets built is, in our experience, the moment this kind of theory stops being abstract.

A third walkthrough runs the Bounded Optimization Explorer. Given an NFA, it enumerates candidate epsilon-edge deletions up to a configured edit depth, checks each one against the baseline NFA for language equivalence using a product-construction procedure, and labels every candidate either equivalent, meaning it is safe to apply and lands on the verified Pareto frontier if it is also not dominated on size, or different, with a concrete shortest string that the baseline accepts and the candidate rejects (or the other way around). Table 7.3 shows one verified example of each kind, computed directly by the engine, not written up from memory.

Note: we have not yet captured live browser screenshots of this walkthrough, for the reason already given in the List of Figures. The counts and examples above come straight from running the compiled engine against the exact patterns shown.

CHAPTER 7
RESULT AND DISCUSSION

7.1 Pipeline state counts for representative patterns

Table 7.1 – Pipeline state counts (produced by running the compiled native C++ engine directly)

Pattern	NFA states	DFA states	Min-DFA states
`(a|b)*abb`	14	5	4
`a+`	5	3	2
`a?`	4	2	2
`[a-c]+`	5	3	2

7.2 A regression found and fixed during this review, with timing evidence

While reviewing pull request #72, we found that its minimization pass had quietly lost an alphabet equivalence-class compression optimization that already exists on the project's main branch, added earlier for issue #64. In its place sat a flat 5,000,000-unit work budget checked against the **uncompressed** alphabet instead of the compressed one. In practice this meant a perfectly legal pattern, five hundred repetitions of the wildcard `.` sitting right at the 500-character length cap, got wrongly rejected as `AutomatonTooComplexError`. The wildcard's alphabet has 126 symbols, and left uncompressed that pushed the work estimate over budget, even though every one of those 126 symbols collapses into a single equivalence class once you actually compress them. We restored the compression step, which groups alphabet symbols by identical transition behaviour before the work estimate is computed, in both the TypeScript and the C++ minimization passes, and kept everything the pull request itself had added: the configurable `MinimizationLimits` parameter and the shared `AutomatonTooComplexError` type.

Table 7.2 – Minimization stress timings, native (non-WASM) Release build, this report's own measurements, 3 runs each

Pattern	Min-DFA states	Run 1	Run 2	Run 3
`.` × 500 (at the length cap)	501	104.4 ms	96.8 ms	96.6 ms
`\w` × 250 (500 chars)	251	23.5 ms	21.5 ms	22.3 ms
`[^a]` × 125 (500 chars)	126	13.9 ms	13.6 ms	12.8 ms

Table 7.2b – Same three patterns through the rebuilt WASM module, one run each via `scripts/check-wasm-limits.mjs` (spawns a fresh Node child process and instantiates the module per probe, so these include process-startup and WASM-instantiation overhead that Table 7.2's native, in-process numbers do not)

Pattern	Elapsed	Time budget	Result
`.` × 500	426 ms	2000 ms	ok
`\w` × 250	140 ms	1000 ms	ok
`[^a]` × 125	141 ms	1000 ms	ok

Table 7.2's timings cover the full `runPipeline` call, parsing through matching, not minimization in isolation, so they are an upper bound on what minimization alone costs. All three patterns now complete successfully with no `AutomatonTooComplexError`; before the fix, the `.`×500 case was rejected outright. We are not claiming WebAssembly is faster or slower than native here. Table 7.2b's numbers include per-probe process-spawn and module-instantiation overhead that Table 7.2's in-process numbers do not, so the two are not directly comparable to each other, and neither is a benchmark claim on its own.

7.3 Bounded-optimizer safe and unsafe deletion examples

Table 7.3 – Bounded-optimizer safe vs. unsafe epsilon-deletion examples (computed directly by the engine)

Pattern	Candidate edit	Equivalence result	Witness

`a|a`	Delete one of the two branch-entry epsilon edges (`delete-eps:1>5`)	**Equivalent (safe)**	— (no distinguishing input exists; the other branch still reaches accept on `a`)
`(a|b)*ab`	Delete the epsilon edge feeding the mandatory trailing `a` (`delete-eps:7>8`)	**Different (unsafe)**	Shortest witness: `"ab"` – baseline accepts, candidate rejects

The safe example is a genuinely redundant structure. Thompson's construction for `a|a` builds two parallel branches that both recognize the literal string `a`, so deleting either branch's own entry edge into the shared accept state leaves the other branch perfectly intact and the language unchanged. The unsafe example is why the explorer has to verify every candidate instead of trusting a syntactic heuristic. That edge looks structurally similar to the redundant kind, but it actually sits on the only path into the mandatory trailing `a`, so deleting it removes acceptance of every string that needs that `a`. The explorer's product-construction equivalence check catches this and reports the shortest such string, `"ab"`, instead of simply saying "unsafe" and leaving the reader to take it on faith.

7.4 Discrepancies against the project's own report-data file

We checked the project repository's own `data_for_report.md` (dated 25 October 2026) against the numbers we derived independently for this report. The automated test counts (209 Vitest tests across 20 files, 81 Catch2 cases totalling 4,923 assertions) and the bounded-search default limits (12 eligible edges, edit depth 2, 24 candidates, 128 DFA states, 100,000 work units, 10,000 product-state pairs) matched exactly. That file records the compiled-WASM probe count as 27/27, which is what it was before we restored the four previously-deleted timed probes to `scripts/check-wasm-limits.mjs` as part of the §7.2 fix. After that restoration and a full WASM rebuild, the real count is **31/31** (Table 5.4b): the original 27 unchanged plus the 4 we brought back. We also went further than the probe count and confirmed that the minimized-DFA JSON output, in both TypeScript and C++, is byte-identical to the project's `main` branch across the shared parser fixture's 24 valid patterns and 200 randomly generated valid patterns. In other words, the §7.2 fix changes nothing about how any pattern that was already legal gets handled.

7.5 Cost analysis

Table 7.4 – Cost analysis (tooling)

Tool	Licence	Cost
GCC / Clang, CMake	Open source (GPL/BSD-family)	₹0
Node.js, npm, TypeScript	Open source (MIT)	₹0
Vitest, Catch2	Open source (MIT/BSL)	₹0
Emscripten	Open source (MIT/University of Illinois)	₹0
React, Vite	Open source (MIT)	₹0
Deployment (static hosting, e.g. GitHub Pages)	Free tier is sufficient; no server-side regex evaluation required	₹0

Every tool in the stack is free and open source, and the production engine runs entirely client-side as a static WebAssembly asset, so the project's total tooling and hosting cost comes to ₹0, not counting our own time.

CHAPTER 8
CONCLUSION AND FUTURE WORK

When we started, "implement a regex engine" sounded like a weekend project. It was not, and that turned out to be the point. RegexLab shows that the classical automata-theory pipeline named in a compiler-design syllabus, Thompson's construction, subset construction, and DFA minimization, can genuinely be built by hand, in two independent languages, checked by differential testing against industrial-strength reference engines, and shipped as an interactive browser tool, with a bounded and formally verified NFA-structure optimizer on top that never makes a false safety claim. We have tried throughout this report to be honest about where the syllabus coverage runs deep, lexical analysis through the indirect route, and where it stays thin, semantic analysis and parallelism, rather than stretching the project to look broader than it is.

The part of this work we are proudest of is not a feature, it is a process. While reviewing our own pull request history for this report, we found a real regression: a lost alphabet-compression optimization in the minimization pass that was quietly rejecting a pattern it had no business rejecting. Finding it was not luck. It came directly from the correctness strategy the project is built on, two independently maintained implementations and a complexity derivation we could actually check by hand, and that same strategy is what let us fix it and prove the fix with real timing evidence instead of just patching the symptom and hoping. That is, honestly, the most reassuring thing about the whole project: when something did go wrong, the project's own design caught it.

There is plenty we would still like to build. Hopcroft's $O(n \cdot \log n \cdot k)$ minimization alongside Moore's, with a test proving both produce the same minimized automaton. The direct DFA-construction method, nullable/firstpos/lastpos/followpos, as a second and independently verifiable route to the same DFA. An explicit DFA-state-count guard on subset construction, which today relies only on the indirect bound the pattern-length cap happens to provide. And the AI-assisted suggestion layer itself, whose one non-negotiable rule, that any AI-proposed fix has to be re-validated by the same hand-built parser before a user ever sees it, is already decided and written down even though the feature isn't built yet. None of this was rushed, and we would rather hand in a smaller project we can defend completely than a longer feature list we cannot.

CHAPTER 9
REFERENCES

1. K. Thompson, "Programming Techniques: Regular expression search algorithm," *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, 1968.
2. M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.
3. E. F. Moore, "Gedanken-experiments on sequential machines," in *Automata Studies*, Princeton University Press, 1956, pp. 129–153.
4. J. E. Hopcroft, "An n log n algorithm for minimizing states in a finite automaton," Stanford University, Computer Science Technical Report CS-TR-71-190, 1971.
5. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.
6. R. Cox, "Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...)," swtch.com, Jan. 2007. [Online article].
7. I. Baburin and R. Cotterell, "A Close Analysis of the Subset Construction," *arXiv:2407.09891*, 2024.
8. T. Jiang and B. Ravikumar, "Minimal NFA problems are hard," *SIAM Journal on Computing*, vol. 22, no. 6, pp. 1117–1141, 1993.
9. L. Clemente and R. Mayr, "Efficient reduction of nondeterministic automata with application to language inclusion testing," *arXiv:1711.09946*, 2017.

Every reference above was checked directly against its source (publisher listing, arXiv abstract page, or SIGACT/SIAM record) while preparing this report, including the exact volume, issue, and page numbers. Two citations in an earlier draft of this report were wrong in a way worth naming honestly: [7] had been attributed to a different Baburin and Cotterell paper on Glushkov-style constructions, and [9] had been attributed to Holzer, Jakobi, and Kutrib's "Magic Number Problem" paper under the wrong arXiv identifier. Both are corrected above to the papers their cited identifiers actually point to.

APPENDIX A
SAMPLE CODE

The extended grammar implemented by `ParserPass` is shown below. We verified it against the per-rule docblocks that actually sit above each parsing function in `engine/passes/parser-pass.cpp` (for example `/** regex ::= term ('|' term)* */` above `parseRegex()`, and `/** atom ::= ASCII_CHAR | '.' | '[' class ']' | '(' regex ')' | escape */` above `parseAtom()`), and separately against the grammar's own reference document, `docs/regex-grammar-v1.md`.

```
regex          ::= term ('|' term)*
term           ::= factor+
factor         ::= atom quantifier?
quantifier     ::= '*' | '+' | '?'
atom           ::= CHAR | '.' | characterClass | '(' regex ')'
characterClass ::= '[' '^'? classItem+ ']'
classItem      ::= classAtom ('-' classAtom)?
classAtom      ::= ASCII_LITERAL | ESCAPE | SHORTHAND   ( \d \w \s \D \W \S )
```

This is the full extended grammar from the issue #60/#64 follow-up, not the narrower Core-phase grammar (literal, concat, alt, star, parens only) that shipped first, as described in §2.5. `+` and `?` are real postfix quantifiers that produce dedicated `PlusNode`/`OptionalNode` AST nodes, not literal characters, and `.`, character classes, and shorthand classes all desugar to a single `CharacterSetNode` Thompson fragment, which is what §5.2's complexity note on alphabet size is referring to.

The restored alphabet equivalence-class compression step in `MinimizationPass` (TypeScript), central to the fix described in §7.2, copied verbatim from `engine/passes/minimization-pass.ts`:

```
function representativeAlphabetOf(dfa: Automaton, alphabet: string[], stateCount: number):
string[] {
  const classKeyOf = new Map<string, string>(); // symbol -> its class's key (the full transition
column)
  const seenKeys = new Set<string>();
  const representatives: string[] = [];

  for (const symbol of alphabet) {
    const column: string[] = [];
    for (let state = 0; state < stateCount; state++) {
      const dest = dfa.transitions.get(state)?.get(symbol)?.[0];
      column.push(dest === undefined ? "-" : String(dest));
    }
    const key = column.join(",");
    classKeyOf.set(symbol, key);
    if (!seenKeys.has(key)) {
      seenKeys.add(key);
      representatives.push(symbol);
    }
  }

  return representatives;
}
```

The Thompson's-construction rule for alternation (TypeScript; C++ mirrors it exactly), copied verbatim from `engine/passes/thompson-pass.ts`:

```
case "alt": {
  const left = buildFragment(node.left, builder);
  const right = buildFragment(node.right, builder);
  const start = builder.newState();
  const accept = builder.newState();
  builder.addTransition(start, "eps", left.start);
  builder.addTransition(start, "eps", right.start);
  builder.addTransition(left.accept, "eps", accept);
  builder.addTransition(right.accept, "eps", accept);
  return { start, accept };
}
```